

ACTIVIDAD 12: LAB: FILE PATH TRAVERSAL, VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

CNO IV (T37A): Seguridad Informática

Alumno: Alejandro García Morales
Profesor: Mtro. Servando López Contreras

Contenido

1. Nombre del Laboratorio	2
2. Nivel de Dificultad	2
3. Tipo de Explotación	2
4. Objetivo del Laboratorio	2
5. Marco Teórico y Fundamentación Técnica	2
5.1. Definición Formal de la Vulnerabilidad	2
5.2. Mecánica del Null Byte (%00).....	2
5.3. El Fallo Lógico en la Validación de Extensiones.....	3
6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite	3
7. Registro de Evidencias Técnicas	7
8. Explicación Técnica del Payload	7
9. Mitigación Recomendada y Principios de Defensa en Profundidad	8
9.1. Modernización de Entornos de Ejecución.....	8
9.2. Sanitización Estricta de la Entrada (Allowlisting)	8
9.3. Uso de APIs de Obtención de Nombre de Archivo.....	8
10. Referencias bibliográficas.....	8

1. Nombre del Laboratorio

Lab: File path traversal, validation of file extension with null byte bypass.

Plataforma: PortSwigger Web Security Academy.

2. Nivel de Dificultad

Practitioner.

3. Tipo de Explotación

- **Categoría principal:** Inyección de secuencias de escape de directorio (File Path Traversal).
- **Mecanismo:** Evasión de validación de extensiones de archivo mediante la inyección de un Byte Nulo (Null Byte %00), explotando el comportamiento de terminación de cadenas en APIs a nivel de sistema operativo (lenguaje C).

4. Objetivo del Laboratorio

La aplicación valida estrictamente que el nombre de archivo suministrado en la petición termine con una extensión de archivo esperada (por ejemplo, .jpg o .png). El objetivo es evadir este filtro agregando la extensión requerida al final del payload, pero separándola de la ruta maliciosa utilizando un byte nulo (%00), lo que permite truncar la cadena a nivel de sistema operativo y recuperar exitosamente el archivo /etc/passwd.

5. Marco Teórico y Fundamentación Técnica

5.1. Definición Formal de la Vulnerabilidad

Esta vulnerabilidad combina Path Traversal (CWE-22) con una debilidad clásica de manejo de cadenas conocida como inyección de Byte Nulo (CWE-626: Null Byte Injection). Ocurre cuando componentes de software de alto nivel interactúan con funciones nativas del sistema operativo escritas en C/C++.

5.2. Mecánica del Null Byte (%00)

En los lenguajes de programación basados en C, las cadenas de texto no tienen un atributo de longitud predefinido; en su lugar, utilizan un carácter de terminación explícito para indicar dónde termina la cadena en la memoria. Este carácter es el byte nulo, representado en ASCII como NUL y en codificación URL como %00. Cuando una API nativa del sistema de archivos lee una ruta, procesará los caracteres de izquierda a derecha. Al encontrar un byte nulo (%00), asumirá inmediatamente que la cadena ha terminado, descartando completamente cualquier texto que se encuentre a la derecha del mismo.

5.3. El Fallo Lógico en la Validación de Extensiones

El atacante diseña un payload como `archivo.txt%00.jpg`.

1. **Capa de Validación (Alto Nivel):** El servidor web usa funciones seguras respecto a bytes nulos. Al leer toda la cadena, confirma que termina en `.jpg` y permite que continúe el proceso.
2. **Capa de Ejecución (Bajo Nivel):** El servidor pasa la cadena al sistema operativo para abrir el archivo. El kernel lee `archivo.txt`, encuentra el `%00`, e ignora el resto (`.jpg`). El resultado es que el archivo abierto es `archivo.txt`, logrando una evasión total del filtro de extensiones.

6. Procedimiento Paso a Paso y Uso Técnico de Burp Suite

Paso 1: Acceso e inicialización del entorno del laboratorio

Se ingresa a PortSwigger Web Security Academy y se navega a la sección del laboratorio "File path traversal, validation of file extension with null byte bypass" (Nivel Practitioner). Se inicializa el entorno de pruebas haciendo clic en "Access the lab".

Lab: File path traversal, validation of file extension with null byte bypass



This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



Evidencia 01: Interfaz inicial del laboratorio, detallando que la aplicación valida la extensión del archivo al final de la cadena.

Paso 2: Configuración de la interceptación

Se ejecuta Burp Suite Community Edition, se accede a Proxy > HTTP history y se abre el navegador integrado de auditoría ("Open browser") para capturar el tráfico.



Evidencia 02: Ventana principal de Burp Suite con el historial HTTP inicializado y listo para la recolección de tráfico.

Paso 3: Carga de la aplicación objetivo

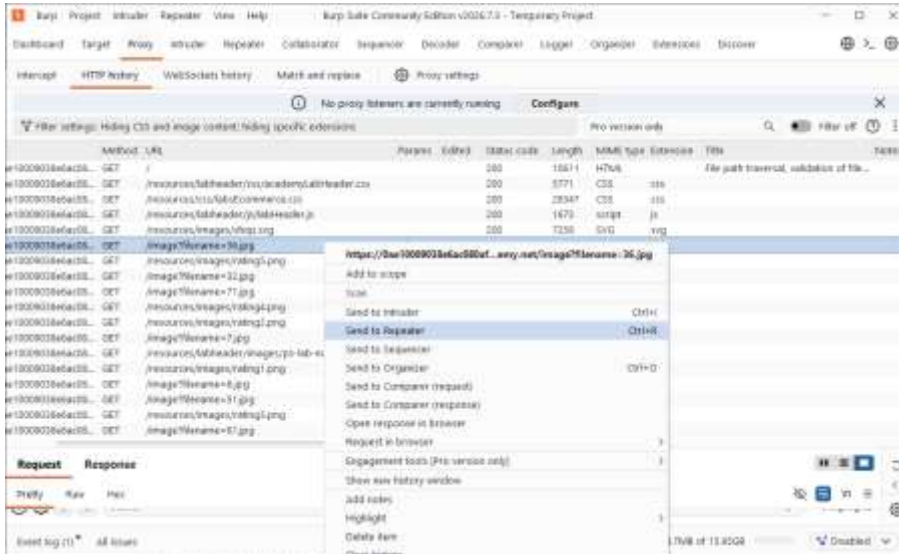
Se visualiza la tienda en línea ("WE LIKE TO SHOP"), la cual renderiza dinámicamente el catálogo de productos y sus imágenes asociadas.



Evidencia 03: Aplicación web renderizada sin bloqueos en el navegador seguro de Burp Suite.

Paso 4: Identificación del parámetro y envío a Repeater

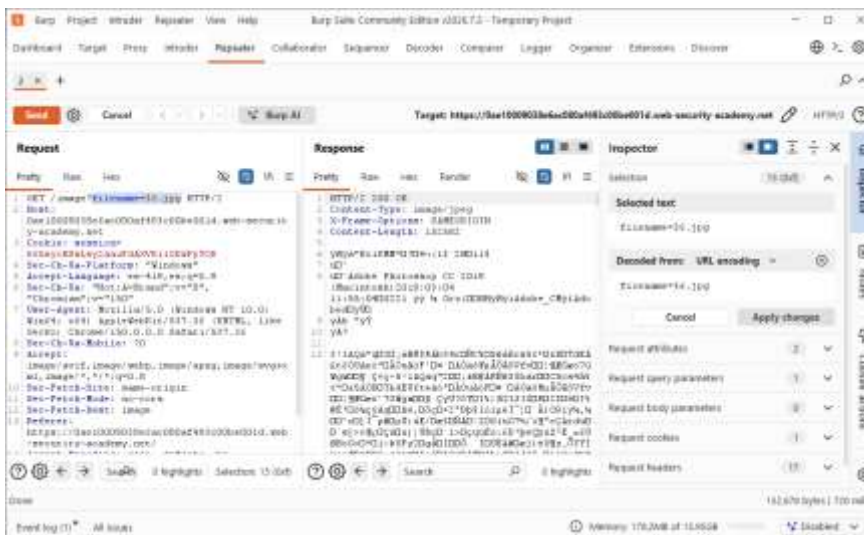
En el historial HTTP de Burp Suite, se localiza la petición GET responsable de solicitar los recursos gráficos: GET /image?filename=36.jpg. Se hace clic secundario sobre esta entrada y se selecciona "Send to Repeater" (Ctrl+R).



Evidencia 04: Historial mostrando la identificación de la solicitud GET hacia filename=36.jpg y su derivación al entorno Repeater.

Paso 5: Comprobación del Comportamiento Base en Repeater

En Repeater, se corrobora que la petición inalterada con la extensión legítima .jpg devuelve exitosamente un HTTP 200 OK con los metadatos de la imagen. Si en este paso se enviara el payload ../../etc/passwd directamente, el servidor devolvería un error, ya que no termina con la extensión autorizada.

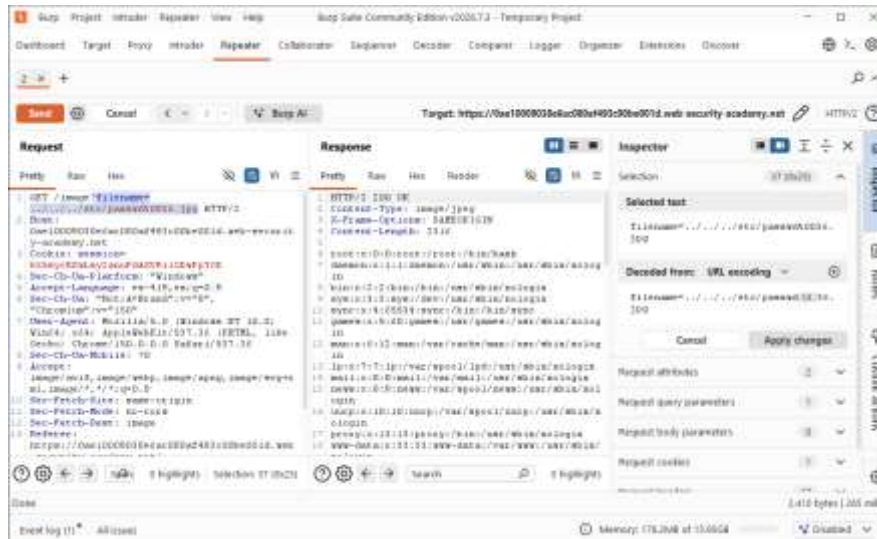


Evidencia 05: Petición original en Repeater mostrando el comportamiento validado de la aplicación al recibir la extensión esperada.

Paso 6: Inyección del Payload con Null Byte y Exfiltración

Para evadir la validación, se construye un payload que incorpora la navegación de directorios, el archivo objetivo, el byte nulo truncador y, finalmente, la extensión requerida por el filtro. Se modifica el parámetro a:

filename=../../../../etc/passwd%0036.jpg. Al enviar la petición, la capa superior de validación verifica que el string finaliza en 36.jpg y lo aprueba. Sin embargo, al pasar a nivel de sistema, el byte %00 trunca la cadena, solicitando únicamente ../../../../etc/passwd. El servidor responde con un **HTTP/2 200 OK**, volcando exitosamente la tabla de usuarios de Linux (root, daemon, bin, etc.).

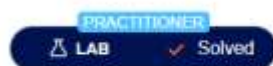


Evidencia 06: Ataque exitoso en Repeater mostrando el payload de Byte Nulo (%00) evadiendo la validación y revelando el contenido de /etc/passwd.

Paso 7: Validación de Compromiso en Plataforma

Se regresa a la ventana principal de PortSwigger en el navegador, la cual confirma la exfiltración de los datos confidenciales actualizando el estado del laboratorio a "Solved".

Lab: File path traversal, validation of file extension with null byte bypass



This lab contains a path traversal vulnerability in the display of product images.

The application validates that the supplied filename ends with the expected file extension.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



Evidencia 07: Confirmación visual del estado "Lab Solved", ratificando la resolución técnica de la vulnerabilidad.

7. Registro de Evidencias Técnicas

ID Evidencia	Elemento Requerido	Descripción Técnica
EVID-01	Inicialización del entorno	Interfaz de inicio de PortSwigger con la premisa de la vulnerabilidad de validación de extensiones.
EVID-02	Configuración del Proxy	Vista de Proxy > HTTP history y preparación del entorno de auditoría.
EVID-03	Aplicación objetivo	Interfaz web de la tienda e-commerce bajo análisis.
EVID-04	Interceptación de tráfico	Identificación de la solicitud GET hacia filename=36.jpg enviada a Repeater.
EVID-05	Comportamiento Base	Repeater procesando correctamente la imagen debido a la extensión válida.
EVID-06	Inyección y Exfiltración	Ejecución del payload <code>../../../../etc/passwd%0036.jpg</code> , provocando el truncamiento nativo y extrayendo los datos.
EVID-07	Acreditación de resolución	Banner de la plataforma indicando "Lab Solved".

8. Explicación Técnica del Payload

Payload Utilizado:

`../../../../etc/passwd%0036.jpg`

Análisis de la Explotación:

1. **Salto de Directorio** (`../../../../etc/passwd`): La primera sección instruye al sistema operativo a salir del directorio público de imágenes, ascender hasta la raíz, e ingresar al archivo de configuración de cuentas.
2. **Null Byte Injection** (`%00`): Representa el carácter hexadecimal de terminación de cadena (NUL byte). Actúa como un marcador de "fin de línea" para las funciones base escritas en C/C++.
3. **Falsa Extensión** (`36.jpg`): Es el cebo para el filtro de la aplicación.
4. **Flujo Crítico:**
 - La aplicación web verifica la cadena completa y confirma que los últimos caracteres son `.jpg`. La prueba pasa.
 - La aplicación envía la cadena a la API del sistema de ficheros.
 - El kernel de Linux procesa `../../../../etc/passwd`. Al toparse con el `%00`, asume que el nombre del archivo terminó ahí. Descarta `36.jpg` de la memoria.
 - El kernel abre y devuelve exclusivamente `/etc/passwd`.

9. Mitigación Recomendada y Principios de Defensa en Profundidad

9.1. Modernización de Entornos de Ejecución

Históricamente, los ataques de Null Byte afectaban masivamente a versiones antiguas de PHP (anteriores a la 5.3.4), Perl y Java. Mantener actualizados los frameworks, los lenguajes de programación del backend y el kernel del sistema operativo es la primera barrera, ya que las versiones modernas implementan protección intrínseca contra este vector, arrojando excepciones si detectan un byte nulo en la ruta de un archivo.

9.2. Sanitización Estricta de la Entrada

No se debe confiar en la validación del final de una cadena (`endsWith()`). Se debe implementar una validación exhaustiva usando expresiones regulares que restrinjan todo el contenido del parámetro a caracteres estrictamente alfanuméricos, bloqueando estructuralmente la inyección de secuencias no imprimibles como `%00` o caracteres de navegación como `/` y `\`.

9.3. Uso de APIs de Obtención de Nombre de Archivo

Antes de cualquier validación, la aplicación debe procesar la entrada mediante una función nativa de extracción que ignore las rutas, como `basename()`. Esto garantiza que, incluso si un atacante intenta inyectar directorios, la aplicación solo operará sobre la última porción de la cadena, neutralizando el recorrido por el sistema de archivos.

10. Referencias bibliográficas

- MITRE Corporation. (2024). *CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- MITRE Corporation. (2024). *CWE-626: Null Byte Injection*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/158.html>
- Open Web Application Security Project. (2021). *A01:2021 - Broken Access Control*. OWASP Top 10. https://top10.owasp.org/2021/A01_2021-Broken_Access_Control/index.html
- PortSwigger. (s. f.). *Lab: File path traversal, validation of file extension with null byte bypass*. PortSwigger Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass>